



Abhinu.Dev Basic ICTL
programming language

Ultimate Guide Booklet



Abhinu Gupta (IndianCoder3)

March 2026

© IndianCoder3 2026-Present

Under the same licensing of the GitHub Project
(IndianCoder3/abhinu-dev_basic-ictl)

This publication may be republished, stored, merged,
or transmitted freely with the below condition:

The above copyright notice and this permission notice
shall be included in all copies or substantial portions
of the Publication.

Introduction

Setting up ICTL on your Computer

1. Please visit https://github.com/IndianCoder3/abhinu-dev_basic-ictl/releases
2. From the top release, download the installer.
3. Run the file and proceed with the install.
4. After installation, open a new terminal instance (either CMD/PS) and type “ictl”, and make sure it’s installed!

Setting up ICTL VS Code Extension

1. Please visit https://github.com/IndianCoder3/abhinu-dev_basic-ictl/releases
2. Download the .vsix file from the top release.
3. Now, open VS Code, and press Ctrl + Shift + P to open the Command Palette.
4. Search for “vsix”. An option will appear, something named like “Extensions: Install from VSIX...” Press Enter, and select the vsix file you downloaded earlier.
5. Now, check if it was installed correctly, by opening a ICTL file.

1

Introduction To ICTL - A Programming Language

Bob was sitting bored – staring on his screen after learning Scratch, but he wanted to now learn a text-based programming language. His friend Toto said to him, “Try ICTL! It’s very easy.” Bob was confused, and says “How do I learn ICTL?”. He replies, “It’s easy! Let me guide you!”.

Let’s join Bob and Toto in their learning journey!



What Will We Learn?

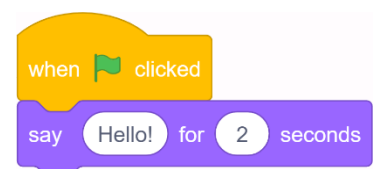
- What is ICTL?
- What is the difference between Scratch and ICTL?
- Why prefer ICTL?
- What can be made with ICTL?

What is ICTL?

ICTL (Abhinu.Dev Basic ICTL) is a beginner friendly text-based programming language, designed to teach you the fundamentals of coding without overwhelming complexity. It was made by Abhinu Gupta (IndianCoder3) on 5th March, 2026. It follows a simple syntax which is easy to remember.

What is the difference between Scratch and ICTL?

Scratch, also known as the block-based programming language, is considered to be the easiest programming language to learn. ICTL is a text-based programming language, hence they are different, though they both are easy. The picture to the right is how you print out “Hello!” in Scratch.



Now, see how you print out “Hello!” in ICTL:

```
Program.Main {  
    Terminal.Echo("Hello!")  
}
```

Did you notice, that both are similar logically? ‘When  clicked’ and ‘Program.Main’ both represent the start of the program.

Why Prefer ICTL?

ICTL is made to make learning text-based programming easy. ICTL **follows the same concept that other programming languages follow** – I/O, Logic, Math, Loops, Variables, etc. When you learn other programming languages afterwards, such as Python, JavaScript, or Java, you already know the basics!

Think of programming languages like spoken languages - they have their own grammar and vocabulary. ICTL is like learning English instead of trying to learn Gujarati, Bengali, and Marathi at the same time. It keeps things simple so you can focus on **learning how to think like a programmer**.

What can be made with ICTL?

By now, you must be curious, “What can this language do?” See this:

Code:

```
Program.Main {  
    Terminal.Echo("Welcome to ICTL!")  
  
    Variables.New(Name)  
    Variables.Name = Terminal.Ask("What is your name? ")  
  
    Terminal.Echo("Nice to meet you, " + Variables.Name + "!")  
}
```

Output:

```
Welcome to ICTL!  
What is your name? Aarush  
Nice to meet you, Aarush!
```

(In the above output, **black** represents the output, and **blue** represents our input.)

As you see, the program greeted us, asked our name, and reused our name in another response. This is all done easily using just Terminal and Variables. This was just an example, ICTL is even more capable!

2

Getting Started - ICTL Exploration and Syntax

Note: It is a good time to read the Introduction, at the beginning of the book.

Toto was just passing by Bob – and finds him struggling with ICTL. He asks “What happened, Bob?”. He replies “I can’t understand ICTL, no one told me about how to write it.” Toto decides to help Bob explore ICTL and know about its syntax, skeleton, and commands. Let’s join their adventurous journey!

What Will We Learn?

- Exploring ICTL
- ICTL Syntax
- ICTL Skeleton
- ICTL Code and Commands
- Code fixing
- Writing your first program

Exploring ICTL

Let’s explore ICTL!

Experiment Time!

Open Notepad on your computer and type in the following code:

```
Program.Main {  
    Terminal.Style("Red")  
    Terminal.Echo("My name is Bob")  
}
```

Run the program (refer to “Introduction” page for how to) and see what happens.

Now, change Red to your favourite colour, and see what happens.

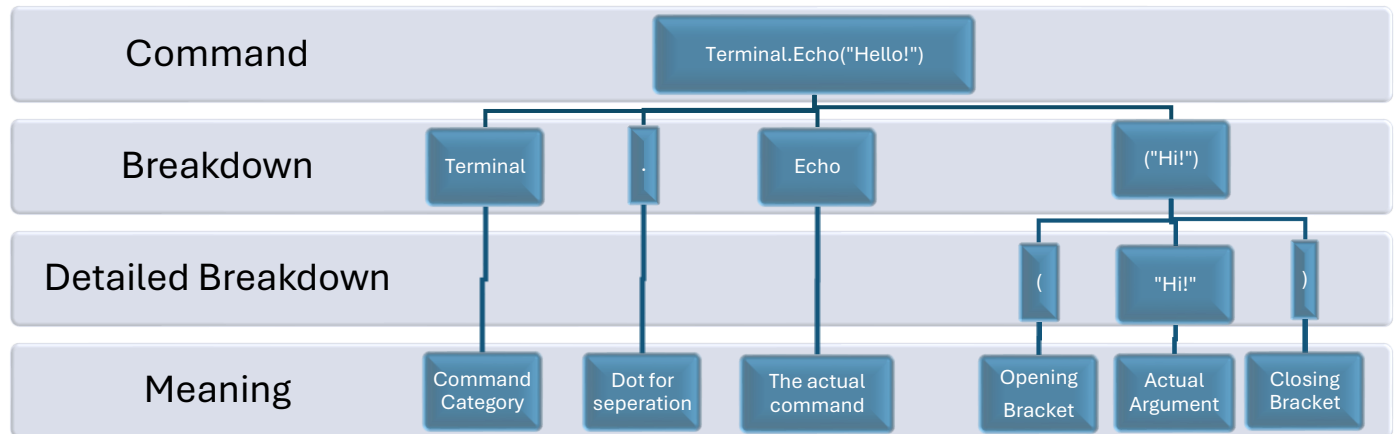
Next, change the name from Bob to your name and see what happens.

Now, what do you think style does? Also, what do you think Terminal.Echo() does?

Let's learn about ICTL, starting from its Syntax.

ICTL Syntax Basics

Each Command is broken into 2 or 3 parts:



Now, you may also want to see an example program:

```
Program.Main {  
    Terminal.Echo("Welcome to ICTL!")  
  
    Variables.New(Name)  
    Variables.Name = Terminal.Ask("What is your name? ")  
  
    Terminal.Echo("Nice to meet you, " + Variables.Name + "!")  
    Terminal.Echo("2+2=" + Math.Eval(2+2))  
}
```

Notice, how each word is starting with a capitalized letter? Yes – ICTL is case sensitive. It needs this in the Command Category, and the Actual Command. The content inside curved parentheses in each command is actually decided by the code writer. You may notice, that arguments use curved parentheses – and that is actually correct, as using any other bracket for arguments will cause a Syntax Error.

Now, let's cover how would you start your program. This is done by Program.Main, as everything inside its curly parentheses is executed during run.

If you want to join (concatenate) 2 or more things together, use the +. For example, to join "Hello " and "World!" together, you will write:

```
"Hello" + "World!"
```

And in a full command:

```
Terminal.Echo("Hello" + "World!")
```

This does not work with numbers, as 2+2 will return 22. This is because math is supposed to be done using Math.Eval().

ICTL Skeleton

Let's refer to the code example on the previous page. See how code is nested within the parentheses and it is indented (having some space from the margin, done using Tab key)? If we nest code further, we have to nest it in parentheses, and it should be indented too. Now, see how arguments rest too in parentheses but curved ones? Inside, the text is in double quotes (not 2 single ones but a double one), and numbers are directly written, without quoting. Now, you may say, that we typed Variables.Name without quotes. This is because that's a function, not a string we want to display.

ICTL Code and Commands

ICTL has many Categories. As per v1.0.0s, they are:

- Terminal
- Variables
- Math
- Data
- Program
- Comments (denoted by both # and //)

Let's cover each of them!

Terminal

Terminal, also known as Console in other languages, is where our input and output is handled. It displays text, and takes input too.

Command	Uses	Example	Command
Terminal.Echo(<string>)	Print what it is said to print out	Terminal.Echo("Hi, " + Variables.Name + " !")	Terminal.Echo(<string>)
Terminal.Ask(<string>)	Take input for what it is said to ask	Variables.Name = Terminal.Ask("Who are you?")	Terminal.Ask(<string>)
Terminal.Style(<style>)	Change the terminal style to whatever is said	Terminal.Style("red")	Terminal.Style(<style>)

There are many styles, refer to the side image for seeing its looks:

Red	Yellow	Green	Blue
Cyan	Magenta	Reset	Bold



```
This is red text!  
This is the same color, but bold!  
This is yellow text!  
This is the same color, but bold!  
This is green text!  
This is the same color, but bold!  
This is blue text!  
This is the same color, but bold!  
This is cyan text!  
This is the same color, but bold!  
This is magenta text!  
This is the same color, but bold!  
This is white (normal) text!
```


Variables

Variables are like storage boxes, you can put in whatever you want, the user's name, age, a string, number, anything!

Variables.New(<name>)

Used to declare new variables, such as:

```
Variables.New("Name")
```

Variables.<name>

1. Variables.Name: Reference/read variable value, for example:

```
>>> Variables.New("Age")
>>> Variables.Age = Terminal.Ask("What is your age? ")
What is your age? 11
>>> Terminal.Echo("You are " + Variables.Age + " years old.")
You are 11 years old.
>>>
```

2. Variables.Name = "Arjun": Assign a value to a variable, for example:

```
>>> Variables.New("Day")
>>> Variables.Day = "Friday"
>>> Terminal.Echo(Variables.Day)
Friday
```

Math

Math is of course – math. It includes evaluation (simplification) and comparison.

Math.Eval(<expression>)

- Evaluate mathematical expressions
- Supported operators: +, -, *, /, ()
- Supports: integers, floats, negative numbers
- Example:

```
Math.Eval(10 + 5)
Math.Eval((2 + 3) * 4)
Math.Eval(3.5 + 2.5)
Math.Eval(-10 + 5)
Math.Eval(Variables.X * 2)
```

Math.Compare(<n1>, <sign>, <n2>)

- Compare two numeric values
- Returns: Boolean (true/false)
- Operators: ">", "<", ">=", "<=", "==", "!="
- Example:

```
Math.Compare(10, ">", 5)
Math.Compare(Variables.X, "<=", 100)
```

Data

Data.Compare(args)

- Compare two values for equality (string or number)
- Returns: Boolean (true/false)
- Compares as strings
- Example:

```
Data.Compare("hello", "hello")
Data.Compare(Variables.Input, "yes")
```

Program

Program.Main {...}

The main code, for example:

```
Program.Main {
    Terminal.Echo("Hello World!")
}
```

Program.If(args) {...}

Execute the block if the condition is true, for example:

```
Program.If(Math.Compare(X, ">", 5)) {
    Terminal.Echo("X is greater than 5")
}
```

Program.Loop(args) {...}

A finite loop, for example:

```
Program.Loop(5) {  
    Terminal.Echo("iteration")  
}
```

Program.ForeverLoop(args) {...}

An infinite loop (unless broken with BreakLoop), for example:

```
Program.ForeverLoop {  
    Variables.Counter = Math.Eval(Variables.Counter + 1)  
    Program.If(Math.Compare(Variables.Counter, ">", 10)) {  
        Program.BreakLoop  
    }  
}
```

Program.BreakLoop

Breaks the current loop, refer to the above example.

Program.Continue

Skip to the next loop iteration, for example:

```
Program.Loop(10) {  
    Program.If(Data.Compare(i, 5)) {  
        Program.Continue  
    }  
    Terminal.Echo(i)  
}
```

Comments

Comments are notes in your script, not parsed by the interpreter. It won't be executed, just like they never existed! These can be used to help understand code better, without triggering them as code. We can use either # or // for comments, both work fine. For example:

```
# This is a comment  
Program.Main {  
    // This is also a comment  
    Terminal.Echo("This is not a comment")  
}
```

Fixing Code Errors

See this code:

```
Progam.Main {  
    telminal.echo(Hi!)  
    Variables.name = telminal.ask(What is your name?)  
    telminal.echo(hi + name!)  
}
```

Now, if you run this, you may see:

Invalid block header: 'Progam.Main'

This is because Program was misspelled to Progam. Now, after we fix it:

Unknown command: 'telminal.echo(Hi!)

This is because Terminal was misspelled to telminal, and also, the names are not starting with a capitalised letter, so instead it should be Terminal.Echo. Now, notice how Hi! Wasn't in quotes? We should fix that as well. Also, let's fix Terminal.Ask, and future uses. Now, when we run the program, we see that there is no spacing when we type our name. This is because we didn't add a space before. Also, our name wasn't printed. This is because we use Variables.Name to reference a variable, not directly its name. Also, we never defined the variable Name. After all the fixes:

```
Program.Main {  
    Terminal.Echo("Hi!")  
    Variables.New("Name")  
    Variables.Name = Terminal.Ask("What is your name? ")  
    Terminal.Echo("Hi" + Variables.Name + "!")  
}
```

Output:

```
Hi!  
What is your name? Tota  
Hi Tota!
```

Experiment Time!

Open Notepad and type the following code:

```
Program.Main {  
    Variables.New("Name")  
    Variables.New("Age")  
    Variables.Name = Terminal.Ask("Hi! Who are you?")  
    Variables.Age = Terminal.Ask("Hey " + Variables.Name + "! What is your age?")  
    Program.If(Math.Compare(Variables.Age, "<", 18)) {  
        Terminal.Echo("You are a kid!")  
    }  
}
```

```

Program.If(Math.Compare(Variables.Age, ">=", 18)) {
    Terminal.Echo("You are an adult!")
}
}

```

See the result after you run the program. Now, add styles, that change based on the age. Make sure to reset them when the program starts!

QUIZ WHIZ!

1. (Lab) Write a program that:
 - a. Asks for the user's name
 - b. Asks for his/her favourite animal/bird
 - c. Asks for his/her favourite food
 - d. Asks for his/her favourite colour
 - e. Makes a story with the words (Mad Libs)

Example Story:

One day, a person named Avanish was walking down the street towards his home. Suddenly, he finds a black, injured lion crying for help. He helped the lion and gave it some pizza. It was very happy!

2. (Lab) How will you program a math chatbot? It should ask the user the question, solve it, return the answer, and ask the user if they want to get another question solved.
3. Is it true that you can make videos with ICTL?
 - a. True
 - b. False
4. What does this program do?

```

Program.Main {
    Terminal.Echo("Hi!")
}

```

- a. Tell the time
 - b. Say "Hi"
 - c. Give a Pearl
 - d. Do Nothing
5. True or False: ICTL uses the .ictl extension.
 - a. True
 - b. False
6. How do you make new variables?

3

Writing Scripts in ICTL And Tips

We have by far learnt about ICTL, but now, is the moment we actually write scripts!

Number Guessing Game

```
Program.Main {
  Variables.New("Secret")
  Variables.New("Guess")
  Variables.New("Attempts")
  Variables.New("TooHigh")
  Variables.New("TooLow")

  Terminal.Echo("=== Number Guessing Game ===")
  Terminal.Echo("I'm thinking of a number between 1 and 100")
  Terminal.Echo("")

  Variables.Secret = 42
  Variables.Attempts = 0

  Program.ForeverLoop {
    Variables.Guess = Terminal.Ask("Make a guess: ")
    Variables.Attempts = Math.Eval(Variables.Attempts + 1)

    # Check if correct
    Program.If(Math.Compare(Variables.Guess, "=", Variables.Secret)) {
      Terminal.Echo("You got it!")
      Terminal.Echo("It took " + Variables.Attempts + " attempts!")
      Program.BreakLoop
    }

    # Check if too low
    Program.If(Math.Compare(Variables.Guess, "<", Variables.Secret)) {
      Terminal.Echo("Too low!")
    }

    # Check if too high
    Program.If(Math.Compare(Variables.Guess, ">", Variables.Secret)) {
```

```

    Terminal.Echo("Too high!")
  }
}
}

```

Now, let's make some changes!

1. See at the top where we define our secret number, 42. Change it to anything you like between 1 to 100.
2. If you want, you can change the range in the text too! Just make sure the range has the number.

Temperature Converter

```

Program.Main {
  Variables.New("Celsius")
  Variables.New("Fahrenheit")
  Variables.New("Choice")

  Terminal.Echo("=== Temperature Converter ===")
  Terminal.Echo("1. Celsius to Fahrenheit")
  Terminal.Echo("2. Fahrenheit to Celsius")

  Variables.Choice = Terminal.Ask("Choose (1 or 2): ")

  Program.If(Data.Compare(Variables.Choice, "1")) {
    Variables.Celsius = Terminal.Ask("Enter temperature in Celsius: ")
    # Formula: F = (C * 9/5) + 32
    Variables.Fahrenheit = Math.Eval((Variables.Celsius * 9 / 5) + 32)
    Terminal.Echo(Variables.Celsius + "°C = " + Variables.Fahrenheit + "°F")
  }

  Program.If(Data.Compare(Variables.Choice, "2")) {
    Variables.Fahrenheit = Terminal.Ask("Enter temperature in Fahrenheit: ")
    # Formula: C = (F - 32) * 5/9
    Variables.Celsius = Math.Eval((Variables.Fahrenheit - 32) * 5 / 9)
    Terminal.Echo(Variables.Fahrenheit + "°F = " + Variables.Celsius + "°C")
  }
}

```

Now, compare the program's answers, and the answers that you get when solving by yourself!

Grade Calculator

```
Program.Main {
    Variables.New("Score")
    Variables.New("MkScored")
    Variables.New("MkCap")
    Variables.New("Grade")

    Terminal.Style("reset")
    Terminal.Style("magenta")
    Terminal.Echo("=====GRADE CALCULATOR=====")

    Variables.MkScored = Terminal.Ask("Enter your score: ")
    Variables.MkCap = Terminal.Ask("Enter the max cap: ")

    Variables.Score = Math.Eval((Variables.MkScored/Variables.MkCap)*100)
    # Assign grades
    Program.If(Math.Compare(Variables.Score, ">=", 80)) {
        Terminal.Style("green")
        Variables.Grade = "A"
    }
    Program.If(Math.Compare(Variables.Score, ">=", 60)) {
        Program.If(Math.Compare(Variables.Score, "<", 80)) {
            Terminal.Style("cyan")
            Variables.Grade = "B"
        }
    }
    Program.If(Math.Compare(Variables.Score, ">=", 40)) {
        Program.If(Math.Compare(Variables.Score, "<", 60)) {
            Terminal.Style("blue")
            Variables.Grade = "C"
        }
    }
    Program.If(Math.Compare(Variables.Score, ">=", 30)) {
        Program.If(Math.Compare(Variables.Score, "<", 40)) {
            Terminal.Style("yellow")
            Variables.Grade = "D"
        }
    }
    Program.If(Math.Compare(Variables.Score, "<", 30)) {
        Terminal.Style("red")
        Variables.Grade = "F"
    }
}
```



```

# Result
Terminal.Echo("=====RESULT=====")
Terminal.Echo("Score: " + Variables.Score + "%")
Terminal.Echo("Grade: " + Variables.Grade)

Program.If(Data.Compare(Variables.Grade, "A")) {
Terminal.Echo("Excellent work!")
}

Program.If(Data.Compare(Variables.Grade, "B")) {
Terminal.Echo("Good Work! Just a bit more and you were golden!")
}

Program.If(Data.Compare(Variables.Grade, "C")) {
Terminal.Echo("Average Grade...")
}

Program.If(Data.Compare(Variables.Grade, "D")) {
Terminal.Echo("Below Average, you could have done more!")
}

Program.If(Data.Compare(Variables.Grade, "F")) {
Terminal.Echo("You didn't study, right?")
}

Terminal.Style("reset")
}

```

Tips

1. Plan First: Plan what would your program do.
2. Declare Variables Early: Define them at the start of the program.
3. Comment your code: A good practice for future visits to the same code.
4. Test everything: Try different inputs and edge cases.
5. Name variables clearly: Use UserAge as the name, not UA
6. One task per variable: Don't use a variable meant for storing name for everything like age!

Do It Yourself!

1. Make a program that:
 - a. Asks for 3 test scores, the max in each, and the scored marks in each
 - b. Calculates the average
 - c. Determine the average as A, B, C, D, E, or F
 - d. Displays the results in a nice format.
2. Make a program that asks the users how many numbers will they enter (with a max cap of 10), ask them that times a number, and calculate their average.